

Pandas Basics & Data Cleaning

Read / Write

```
import pandas as pd
df = pd.read_csv('file.csv', index_col=0)
df.to_csv('out.csv', index=False)
```

Inspect

```
df.shape      # (rows, cols)
df.dtypes     # column types
df.info()     # summary
df.describe() # stats (numeric)
df.describe(include='all') # +object
df.head(5); df.tail(3); df.sample(3)
df.columns    # column names
df['col'].unique() # unique values
df['col'].nunique() # count unique
df['col'].value_counts(normalize=False, dropna=False) # count null as a value
df.select_dtypes(include='object') # can change include to 'number'
```

Filter / Sort

```
df[df['col'].isin(['A','B'])] # Filter
# Sort
df.sort_values('col')
df.sort_values('col', ascending=False)
df.sort_values(['c1','c2'], ascending=[True,False])
```

Create / Modify Columns

```
df['new1'] = df['a'] * df['b']
df['new1'] = df['col'].apply(lambda x: x**2)
df.drop(columns=['col'])
df.rename(columns={'old':'new'})
```

Missing Data

```
df.isnull().sum() # count NaN per col
df.dropna()      # drop rows with NaN
df.dropna(subset=['col1']) # drop if col is NaN
df.fillna(0)      # fill all with 0
df.fillna({'col1':'X', 'col2': df['col2'].mean()})
df['col'].interpolate() # linear interpolate
```

GroupBy & Agg

```
df.groupby('col')['val'].mean()
df.groupby('col')['val'].mean().reset_index(name='avg')
df.groupby('col').agg(['min','max','mean'])
df.groupby('col').size().reset_index(name='count')
```

```
groupby_df = vdo_df.groupby(['trending_date', 'types'])['views'].sum()
unstacks = groupby_df.unstack('types', fill_value=0)
```

Merge / Concat

```
pd.concat([df1, df2]) # stack rows
pd.concat([df1, df2], axis=1) # stack cols
df.merge(df2, on='key', how='inner')
# how: 'inner', 'left', 'right', 'outer'
```

Pivot / Melt

```
pivot = df.pivot_table(index='row', columns='col',
                        values='val', aggfunc='mean')
melt = pd.melt(df, id_vars=['id'],
               value_vars=['c1','c2'])
```

VIZ1: Streamlit + Map (pydeck)

Streamlit Basics

```
import streamlit as st
st.set_page_config(page_title="X", layout="wide")
# layout: "centered" | "wide"
st.title('Title'); st.header('H1'); st.subheader('S')
st.write('Text'); st.text('plain')
st.dataframe(df); st.table(df)
```

Sidebar Filters

```
st.sidebar.header("Filters")
selected = st.sidebar.multiselect("Label",
    options=df['col'].unique().tolist(),
    default=df['col'].unique().tolist())
val = st.sidebar.slider("Label",
    min_value=0.001, max_value=0.005, value=0.002, step=0.001, format="%3f" (ตัวจำกัดว่า 3 หลักต้องใส่ format))
choice = st.sidebar.selectbox("Label", ['A','B'])

# Apply
filtered = df[(df['col'].isin(selected)
    & (df['rating'] >= val))]
```

pydeck Map

```
import pydeck as pdk
scatter = pdk.Layer("ScatterplotLayer" (Change to "HeatmapLayer" ได้), data=df,
    get_position=[ 'longitude', 'latitude'],
    get_color='color', # column name (str)
    get_radius=100, pickable=True)
hex_layer = pdk.Layer("HexagonLayer", data=df,
    get_position=[ 'longitude', 'latitude'],
    radius=200, elevation_scale=4, extruded=True)
view = pdk.ViewState(latitude=13.76,
    longitude=100.56, zoom=12, pitch=0)
st.pydeck_chart(pdk.Deck(layers=[scatter],
    initial_view_state=view, map_style='light',
    tooltip=[ "text":(name|n|price), "bah!"]
), height=400)

| tooltip: {col_name} แทนค่า | get_color ใช้ string ชื่อ column
```

Color by column

```
def c(p):
    if p<=500: return [0,200,0,180]
    elif p<=2000: return [255,165,0,180]
    else: return [255,0,0,180]
df['color'] = df['price'].apply(c) #get_color='color'
```

VIZ2: Streamlit + Graph (Plotly)

Plotly Express

```
import plotly.express as px
fig=px.bar(df,x='cnt',y='cat',orientation='h',title='T')
fig=px.line(df, x='month', y='sales')
fig=px.scatter(df,x='p',y='r',color='cat',size='cnt')
fig=px.pie(df, names='cat', values='cnt')
fig=px.histogram(df, x='price', nbins=20)
st.plotly_chart(fig, use_container_width=True)
```

st.columns + st.metric

```
col1,col2,col3 = st.columns(3) # or st.columns([1,1,1])
with col1: st.metric("Total", len(df))
with col2: st.metric("Avg", round(df['p'].mean(),2))
with col3:
    top=df.loc[df['rating'].idxmax(), 'name']
    st.metric("Top", top)
```

Built-in Charts & Run

```
st.bar_chart(df.set_index('cat')['cnt'])
st.line_chart(df.set_index('m')['sales'])
st.map(df) # needs 'latitude' & 'longitude'

# Run: streamlit run student_st.py
```

DE1: Web Scraping (BeautifulSoup)

Setup

```
from bs4 import BeautifulSoup
with open('path','r',encoding='utf-8') as f:
    html = f.read()
soup = BeautifulSoup(html, 'html.parser')
```

find / find all

```
tag = soup.find('tag') # first
tags = soup.find_all('tag') # all
soup.find('td', class_='comments')
soup.find_all('a', class_='country-listing')
soup.find('time', attrs={'datetime':'2025-04-13'})
soup.find('a', string='Labour Day') # exact match
soup.select('table.country-table tr') # CSS
```

Navigation

Method	ทิศทาง	คืน
find_parent('tr')	ขึ้น	1 ตัว
find_parents('tr')	ขึ้น	list
find_next_sibling('tr')	ถัดไป (level เดียว)	1 ตัว
find_next_siblings('tr')	ถัดไป (level เดียว)	list
find_previous_sibling('tr')	ก่อน (level เดียว)	1 ตัว
find_next('td')	ถัดไป (ทุก level)	1 ตัว
find_previous('time')	ก่อน (ทุก level)	1 ตัว

| sibling=ระดับเดียวกัน, next/previous=ข้ามระดับได้

Extract & Patterns

```
tag.text; tag.text.strip(); tag['href']
tag['datetime']; tag.get('title',''); tag.name

# Loop table rows
rows = soup.find('table').find_all('tr')[1:]
for r in rows:
    cols = r.find_all('td')
    day=cols[0].text; date=cols[1].find('time')['datetime']

# find_parent pattern
a=soup.find('a',string='X'); row=a.find_parent('tr')
nxt=row.find_next_sibling('tr')
```

DE2: Spark (PySpark DataFrame)

Setup

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (col, count,
countDistinct, sum, avg, max, min, round,
desc, month, abs as spark_abs, expr, lit)
spark = SparkSession.builder.master('local') \
    .appName('App') .getOrCreate()
df = spark.read.parquet(path)
df = spark.read.csv('netflix-rotten-tomatoes-metacritic-imdb.csv', header=True, inferSchema=True)

# เปลี่ยนชื่อ Columns
cols = [col.replace(' ','_')] for col in df.columns
df = df.toDF(*cols)
df.printSchema(); df.show(5)
```

Core Operations

```
# Cast & new column
df=df.withColumn('Price',col('Price').cast('float'))
df=df.withColumn('rev',col('Price')*col('Qty'))
df=df.withColumn('m',month(col('Date')).cast('date'))

# filter/where (same)
df.filter(col('City')== 'BKK')
df.filter(col('City').isin(['BKK','CNK']))

# groupBy + agg (ต้องอยู่ในเมธอด)
df.groupBy('City').agg(
    sum('rev').alias('total'),
    avg('Price').alias('avg'),
    count('*').alias('cnt'),
    countDistinct('CID').alias('uniq')
).orderBy(desc('total')).show()

# orderBy: desc('col') หรือ ascending=False

# Others
df.count(); df.select('c1','c2')
df.distinct().count(); df.limit(10)
val=df.agg(avg('Price')).first()[0] # scalar
```

```
# Aggregate ด้วย dict
group_df = df.groupBy('Director').agg({'Hidden_Gem_Score':'avg'})
```

```
# ตัวอย่างการใช้ functions สามารถใช้ทำ WordCount ได้
from pyspark.sql import functions
print(df.filter(df['Genre']!=Null()).count())
print(df.filter(df['Genre']!=Null()).select('Genre').withColumn('Genres_split', functions.split(functions.col('Genre'),
    ',' ,)).withColumn('Genre_explode', functions.explode(functions.col('Genres_split'))).groupBy('Genre_explode').agg(
    (('Genre':'count')).count())
```

| .show() คืน None ห้าม assign | .alias() ใช้กับ column ไม่ใช่ groupBy

UDF

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType, IntegerType
```

```
# แบบ lambda (สั้น)
f = udf(lambda x: int(x/5.7)), IntegerType())
df.withColumn('m', f(col('Date')))
```

แบบ function (หากรับ return)

```
def price_tier(price):
    if price is None:
        return 'Unknown'
    elif price < 500:
        return 'Cheap'
    elif price < 2000:
        return 'Mid'
    else:
        return 'Expensive'
```

```
tier_udf = udf(price_tier, StringType())
df = df.withColumn('tier', tier_udf(col('Price')))
```

DE2: Spark RDD

Create & Basic

```
# แปลงจาก Spark DataFrame เป็น rdd ได้
rdd = df.filter(df['Age']!=Null()).select('Age').rdd
```

```
sc = spark.sparkContext
rdd = sc.parallelize([1,2,3,4,5])
rdd = sc.textFile('file.txt') # each line = element
```

Transformations (lazy, return new RDD)

```
rdd.filter(lambda x: x > 2)
rdd.map(lambda x: x * 2) # 1-to-1
rdd.flatMap(lambda x: x.split()) # 1-to-many (flatten)
# map vs flatMap:
# map[['a b','c d']] -> [['a','b'],['c','d']]
# flatMap[['a b','c d']-> ['a','b','c','d']]
```

Actions (trigger computation)

```
rdd.count() # จำนวน elements
rdd.first() # element แรก
rdd.collect() # คือทั้งหมดเป็น list
rdd.take(3) # ดึง 3 ตัวแรก
rdd.reduce(lambda a,b: a+b) # รวมทุก element
```

Word Count (MapReduce)

```
lines = sc.textFile('file.txt')
words = lines.flatMap(lambda l: l.split())
counts = words.map(lambda w: (w,1)) \
    .reduceByKey(lambda a,b: a+b)
# reduceByKey: reduce ตาม key เดียวกัน
```

Aggregate

```
rdd.aggregate(zeroValue, seqOp, combOp)
# (sum, count) example:
rdd.aggregate(0.0,
    lambda z,e: z+[0]+e, z[1]+1),
    lambda a,b: (a[0]+b[0], a[1]+b[1]))
# -> (sum_of_all, count_of_all)
```

DS1: Data Preprocessing for ML

```
1 Load Data

import pandas as pd
X_train=pd.read_csv('X_train.csv')
X_test=pd.read_csv('X_test.csv')
y_train=pd.read_csv('y_train.csv').squeeze()
y_test=pd.read_csv('y_test.csv').squeeze()

# .squeeze() แปลง DataFrame 1 col → Series
```

```
2 Define Feature Lists

num_features = ['age', 'income', 'credit_score']
cat_features = ['city', 'gender', 'education']
```

```
3.1 Build class for pipeline

from sklearn.base import BaseEstimator, TransformerMixin
class DropMissing(BaseEstimator, TransformerMixin):
    def __init__(self, threshold=0.5):
        self.threshold = threshold
    def fit(self, X, y=None):
        X = X.copy()
        self.featurein_ = X.columns.tolist()
        vals = X.isnull().sum()
        vals = vals[vals > self.threshold*X.shape[0]]
        self.remove_cols_ = vals.index.tolist()
        return self
    def transform(self, X):
        X = X.copy()
        X = X.drop(self.remove_cols_, axis=1)
        return X
    def get_feature_names_out(self):
        return [col for col in self.featurein_ if col not in self.remove_cols_]
```

```
3 Build Sub-Pipelines

from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (
    StandardScaler, MinMaxScaler, OneHotEncoder)
```

```
# Numerical: เริ่ม missing → scale
num_pipe = Pipeline([
    ('dropMissing', DropWMissing(threshold=0.5)),
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()))

# StandardScaler: mean=0, std=1
# MinMaxScaler: range [0,1]
# strategy: 'mean','median','most_frequent','constant'

# Categorical: เริ่ม missing → encode
cat_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore',
        sparse_output=False))])

# handle_unknown='ignore' → ค่าใหม่ใน test → 0
# sparse_output=False → return array ไม่ใช่ sparse
```

```
4 ColumnTransformer (รวม pipeline)

from sklearn.compose import ColumnTransformer

preprocessor = ColumnTransformer([
    ('num', num_pipe, num_features),
    ('cat', cat_pipe, cat_features)], remainder='passthrough')
preprocessor.set_output(transform='pandas') # ทำให output เป็น DataFrame
+ featurename เช่น num_Age, cat_education_X (ต่างไปจาก Onehot)
```

```
5 Transform Data

X_train_1 = preprocessor.fit_transform(X_train)
X_test_1 = preprocessor.transform(X_test)

# fit_transform = เรียนรู้+แปลง (train เท่านั้น)
# transform = แปลงอย่างเดียว (test เท่านั้น)
```

fit บน train เท่านั้น ห้าม fit บน test → data leakage

```
6 Full Pipeline (preprocessor + model)

# เป็เ็น make_pipeline โดยไม่ต้องใส่ fit แต่เอา tuple transformer ต่อๆกันไม่ต้องใส่ชื่อ pipe ย่อย
full_pipe = Pipeline([
    (preprocessor, preprocessor),
    ('model', RandomForestRegressor(random_state=42))])
full_pipe.set_output(transform='pandas') # ทำให output เป็น DataFrame
full_pipe.fit(X_train, y_train)
y_pred = full_pipe.predict(X_test)
```

DS2: Prediction Model

```
1 Choose Model

# --- Regression ---
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
m=RandomForestRegressor(n_estimators=200,
    max_depth=5, random_state=42)

# --- Classification ---
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
m=LogisticRegression(random_state=99,max_iter=1000,
    solver='liblinear') # supports f1 & f2
```

```
2 Fit & Predict

m.fit(X_train, y_train)
y_pred = m.predict(X_test)
```

```
3 GridSearchCV (หา hyperparameter ดีสุด)

from sklearn.model_selection import GridSearchCV
param_grid = {
    'model__n_estimators': [100, 200],
    'model__max_depth': [5, 10]}

# param name: 'step_name__param' (double __)
grid=GridSearchCV(full_pipe, param_grid,
    cv=3, scoring='r2', n_jobs=-1)
grid.fit(X_train, y_train)
grid.best_params_ # best hyperparams
grid.best_score_ # best CV score
grid.best_estimator_ # fitted best model
y_pred=grid.predict(X_test)
```

```
4a Evaluate — Regression

from sklearn.metrics import (r2_score,
    mean_absolute_error, mean_squared_error)
r2=r2_score(y_test,y_pred)
mae=mean_absolute_error(y_test,y_pred)
mse=mean_squared_error(y_test,y_pred)**0.5
ae=abs(actual-predicted) # single value
```

Metric	คำตี	ใช้เมื่อ
R2	ใกล้ 1.0	อธิบาย variance ได้กี่ %
MAE	ยังน้อยยิ่งดี	ค่าผิดพลาดเฉลี่ย
RMSE	ยังน้อยยิ่งดี	ลงโทษ error ใหญ่มากกว่า MAE

```
4b Evaluate — Classification

from sklearn.metrics import (accuracy_score,
    f1_score, precision_score, recall_score,
    classification_report, confusion_matrix)
acc=accuracy_score(y_test,y_pred)
f1=f1_score(y_test,y_pred)

# average='binary'(default),'macro','weighted'
print(classification_report(y_test,y_pred, output_dict=True))
cm=confusion_matrix(y_test,y_pred) #[[TN,FP],[FN,TP]]
```

Metric	สูตร	ใช้เมื่อ
Accuracy	(TP+TN)/All	class balance ดี
Precision	TP/(TP+FP)	ห้าม false positive
Recall	TP/(TP+FN)	ห้าม miss positive
F1	2*P*R/(P+R)	balance P & R

Scoring & Hyperparameters	
Task	scoring=
Regression	'r2', 'neg mean absolute error', 'neg mean squared error', , ถ้าใช้ทั้ง 11 และ 12 เปลี่ยนไปใช้ ElasticNet ปรับ alpha กับ l1
Classification	'accuracy', 'f1', 'f1_macro', 'precision', 'recall', 'roc_auc', 'f1_weighted'
Model (Supervised)	Parameters
RandomForest	n estimators, max depth, min samples_split, random_state, parameter ทั่วไปอื่นๆ
LogisticReg	C, penalty('l1'(lasso),'l2'(ridge)), solver, max_iter
SVC	C, kernel('rbf','linear','poly'), gamma
KNN	n neighbors, weights('uniform','distance')
Neural Network (MLP in sklearn)	learning_rate, hidden units แต่ละ layer such as 5(ชั้นแรก),2(ชั้นสอง)), max epochs, solver='adam'
DecisionTree	max depth, min samples_split, criterion (gini (น้อยยิ่งดี), entropy(เทียบลบฟังก์ชันค่าเอนโทรปี)), max_leaf_nodes, ccp_alpha(regularization)
Model (Unsupervised)	Parameters
Kmeans	k (ใช้ elbow method ได้), distanc func, maximum epochs
DBScan	minPoints, eps, metric (เช่น euclidean)

```
Train/Test Split

from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# stratify=y → รักษาสัดส่วน class
```

```
LabelEncoder / OrdinalEncoder

from sklearn.preprocessing import LabelEncoder, OrdinalEncoder
le = LabelEncoder()
y = le.fit_transform(y) # ['cat','dog'] → [0,1]
le.classes_ # array of labels
le.inverse_transform([0,1]) # กลับเป็น string

oe = OrdinalEncoder(categories=[['S','M','L','XL']])
df[['size']] = oe.fit_transform(df[['size']])
```

```
Save / Load Model

import joblib
joblib.dump(model, 'model.pkl') # save
model = joblib.load('model.pkl') # load

import pickle
with open('m.pkl','wb') as f: pickle.dump(model,f)
with open('m.pkl','rb') as f: model=pickle.load(f)
```

```
Cross Validation

from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5,
    scoring='accuracy')
scores.mean() # avg score
scores.std() # std of scores
```

```
Pandas — String & Type

# String methods
df['col'].str.lower()
df['col'].str.upper()
df['col'].str.strip()
df['col'].str.contains('keyword')
df['col'].str.replace('old','new')
df['col'].str.split(' ', expand=True)
df['col'].str.len()

# Type conversion
df['col'] = df['col'].astype(int)
df['col'] = df['col'].astype(float)
df['col'] = df['col'].astype(str)

df['date'] = pd.to_datetime(df['date'], format='%Y-%m-%d') format จาก ปีคค-แบบเต็ม- เดือน - วัน ถ้า y ตัวหลังจะเป็น คคแบบย่อเช่น 25

df['date'].dt.month # extract month
df['date'].dt.year # extract year
df['date'].dt.day_name() # Monday, etc.
```

```
Pandas — Duplicates & Outliers

# Duplicates
df.duplicated().sum() # count dupes
df.drop_duplicates() # remove
df.drop_duplicates(subset=['col'])

# Outliers (IQR method)
Q1 = df['col'].quantile(0.25)
Q3 = df['col'].quantile(0.75)
IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
df_clean = df[(df['col']>=lower) & (df['col']<=upper)]
```

```
Spark SQL

df.createOrReplaceTempView('sales')
result = spark.sql("""
    SELECT City, SUM(Price*Qty) AS revenue
    FROM sales
    GROUP BY City
    ORDER BY revenue DESC
""")
result.show()
```

```
Spark Join / Union

# Join
df1.join(df2, on='key', how='inner')
df1.join(df2, df1.id==df2.id, 'left')
# how: 'inner','left','right','full'

# Union (stack rows, same schema)
df1.union(df2)
df1.unionByName(df2) # match by column name
```

```
Feature Importance

# After fitting RandomForest
importances = model.feature_importances_
# Get feature names from preprocessor
names = preprocessor.get_feature_names_out()
# Sort
feat_df = pd.DataFrame({
    'feature': names,
    'importance': importances
}).sort_values('importance', ascending=False)
```

```
Python Essentials

{k,v for k,v in zip(keys, vals)}

# Enumerate
for i, val in enumerate(lst):

# Zip
for a, b in zip(lst1, lst2):

# String formatting
fvalue='{x:2f}' # 2 decimal
```

```
CSS Selectors (soup.select)

# Basic Selectors
soup.select('intro') # class: class
soup.select('#firstname') # id: tid
soup.select('p') # element: tag
soup.select('') # all elements

# Combinations
soup.select('n1.n2') # both classes
soup.select('n1.n2') # descendant (space)
soup.select('p.intro') # element with class
soup.select('div, p') # group (multiple tags)

# Hierarchies & Siblings
soup.select('div p') # p inside div (any level)
soup.select('div > p') # p is direct child of div
soup.select('div + p') # p immediately after div
soup.select('p ~ ul') # ul preceded by p (siblings)

# Attribute Selectors
soup.select('[target]') # has attribute 'target'
soup.select('[target="b"]') # attr equals exact value
soup.select('[title!=""]') # attr contains word '!'
soup.select('[lang=en]') # starts with 'en' or 'en-'
soup.select("[a[href^='https']]") # starts with 'https'
```